Actividad 1

Simulación de las puertas lógicas básicas en Vivado

Índice

1. Introducción y objetivos 2
2. Creación del proyecto 3
3. Creación de una plantilla de diseño VHDL 3
4. Estructura de un código en VHDL 7
5. Descripción del circuito en VHDL 9
6. Creación de un banco de pruebas (*test bench*) 12
7. Simulación del circuito 21
8. Cerrar el proyecto y recuperación de ficheros importantes 25
Referencias 27

Actividad 1

Simulación de las puertas lógicas básicas en Vivado

1.- Introducción y objetivos

En esta práctica se va a proceder a describir las puertas básicas en VHDL. A continuación, se generará una simulación del circuito descrito para cotejar sus resultados con las tablas de verdad de estas puertas básicas.

El circuito a diseñar en esta actividad se muestra en la Figura 1. El circuito consta de 2 puertos de entrada (*a* y *b*) y 7 puertos de salida (f_0, f_1, f_2, f_3, f_4, f_5 y f_6). La tabla de verdad de estas funciones lógicas se muestra en la tabla 1.

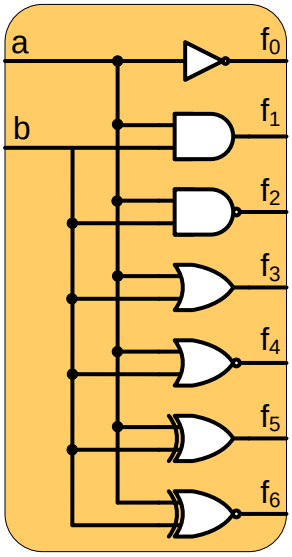


Figura 1. Circuito a diseñar en el proyecto

a	b	f_0	f_1	f_2	f_3	f_4	f_5	f_6
0	0	1	0	1	0	1	0	1
0	1	1	0	1	1	0	1	0
1	0	0	0	1	1	0	1	0
1	1	0	1	0	1	0	0	1

Tabla 1. Tabla de verdad de las funciones lógicas básicas

Los objetivos a alcanzar en esta práctica son:

- Adquirir nociones elementales de VHDL
- Crear una descripción de circuito en VHDL
- Concepto de banco de pruebas o *test-bench*
- Simulación de circuitos basada en *test-bench*

2.- Creación del proyecto

Para la creación del proyecto de Vivado de esta actividad, deben seguirse los pasos descritos en el Apartado 3 de la Actividad 0. En este caso, el nombre del proyecto es *Actividad1* (todo junto) y el resto de parámetros se deja igual.

3.- Creación de una plantilla de diseño VHDL

Una vez creado el proyecto, el siguiente paso consiste en crear un código VHDL con la descripción del circuito que queremos diseñar (el de la Figura 1). Para ello, podemos buscar en la barra de menús *File -> Add Sources*, o, más directamente, pulsar en el comando en el panel *Flow Navigator* (Figura 2).

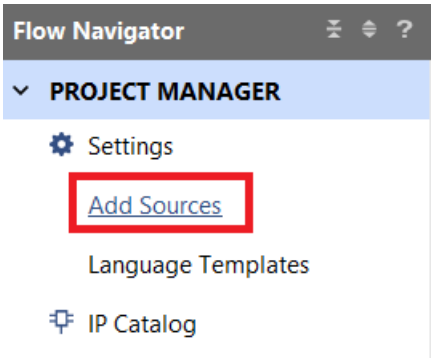


Figura 2. Captura de pantalla con el comando *Add Source* resaltado con cuadro rojo

Aparece un asistente en el que debemos realizar los siguientes pasos:

1. Seleccionar *Add or create design sources* (Figura 3).

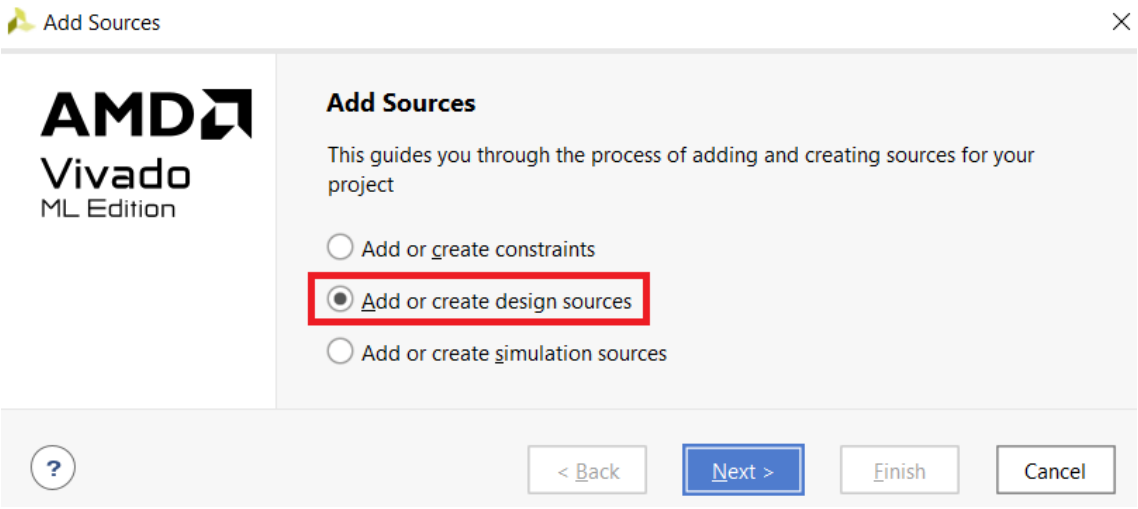


Figura 3. Captura de pantalla de *Add Source* con *Add or create design sources* resaltado con cuadro rojo

2. Seleccionar *Create File* en el paso *Add or Create Design Sources* (Figura 4).

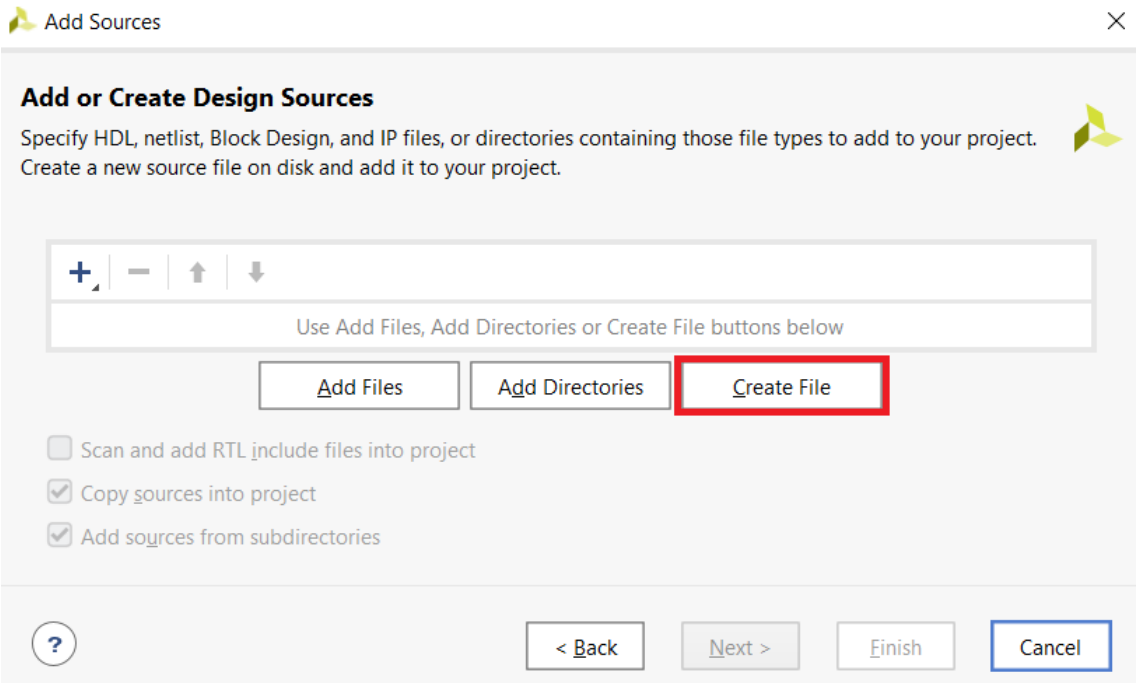


Figura 4. Captura de pantalla de *Add Source* con *Create File* resaltado con cuadro rojo

3. En el siguiente paso del asistente se pone el nombre del fichero VHDL que se va a generar. Lo normal es que el nombre esté asociado la funcionalidad del circuito que se va a proceder a describir (por ejemplo, multiplexor, contador, decodificador, etc.). En este caso, lo llamamos *funciones* (Figura 5) y pulsamos *OK*.

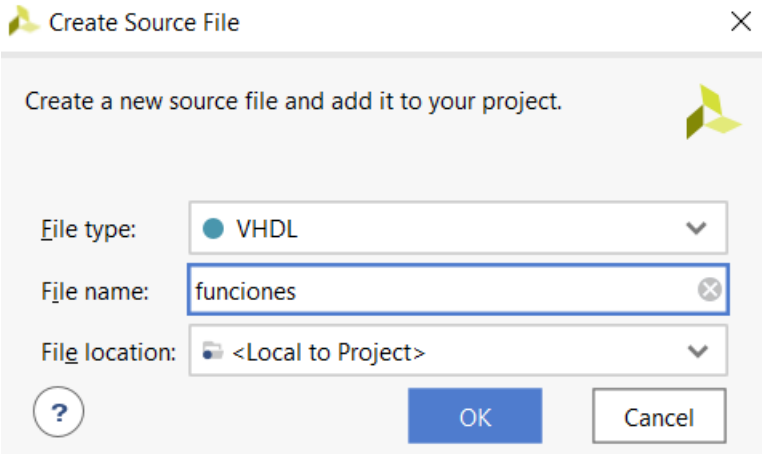


Figura 5. Captura de pantalla de *Create Source File*

4. Comprobamos que el fichero a crear se encuentra añadido en la tabla de *Add or Create Design Sources* y pulsamos *Finish* (Figura 6).

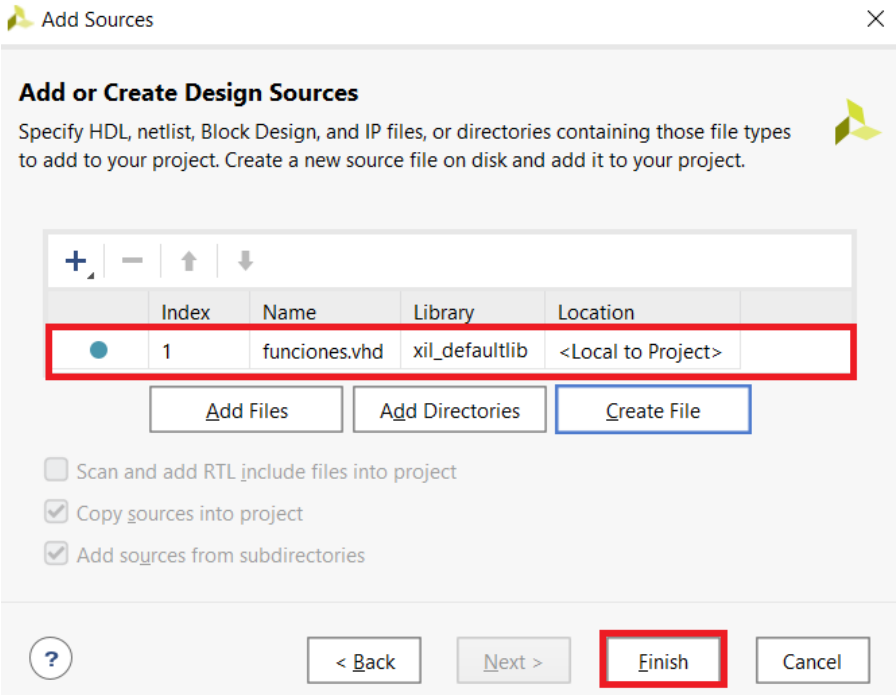


Figura 6. Captura de pantalla de *Add or Create Design Sources* con *Finish* resaltado con cuadro rojo

5. Se rellena la tabla I/O Port Definitions con el nombre de los puertos de entrada y salida del circuito. Se debe indicar también si el puerto es de entrada o de salida (*in* u *out*, respectivamente). **IMPORTANTE**: no modificar el nombre de la entidad (*Entity name*), ya que debe coincidir con el nombre del fichero.

Tras introducir los datos de la interfaz del circuito, pulsamos en OK (Figura 7).

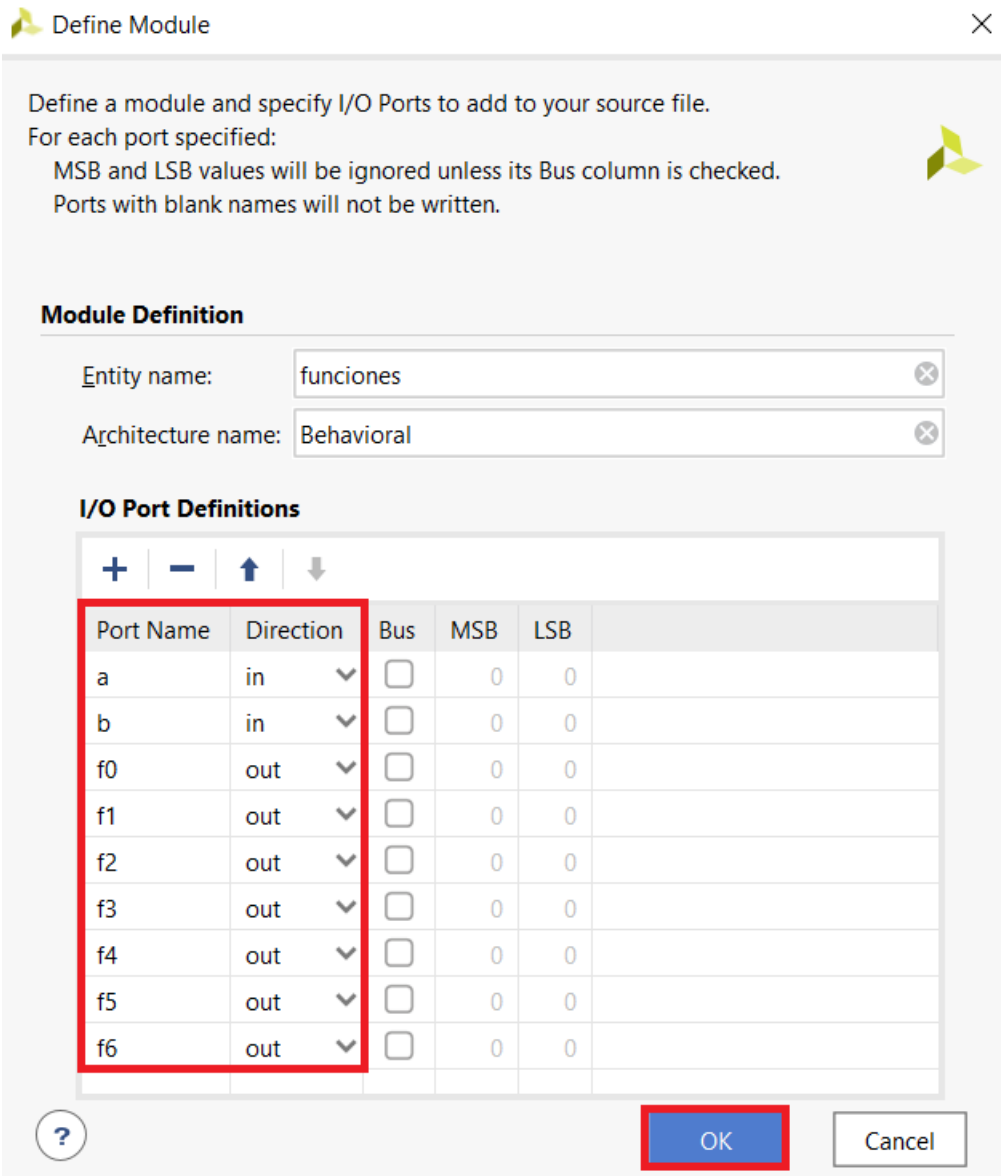


Figura 7. Captura de pantalla de *Define Module* con *OK* resaltado con cuadro rojo

Una vez completados los pasos del asistente, podemos observar en la ventana *Sources* el fichero generado (Figura 8). El icono nos indica que el circuito descrito en este código VHDL es el *top level*, esto es, el circuito que se va a implementar en la FPGA, ya que todo proyecto puede contener más de un circuito. Incluso el circuito *top level* puede estar conformado por otros subcircuitos descritos en otros ficheros. En negrita está el nombre de la entidad, entre paréntesis el nombre de la arquitectura en primer lugar (*Behavioral*) y el nombre del fichero donde se encuentra el circuito (*funciones.vhd*).

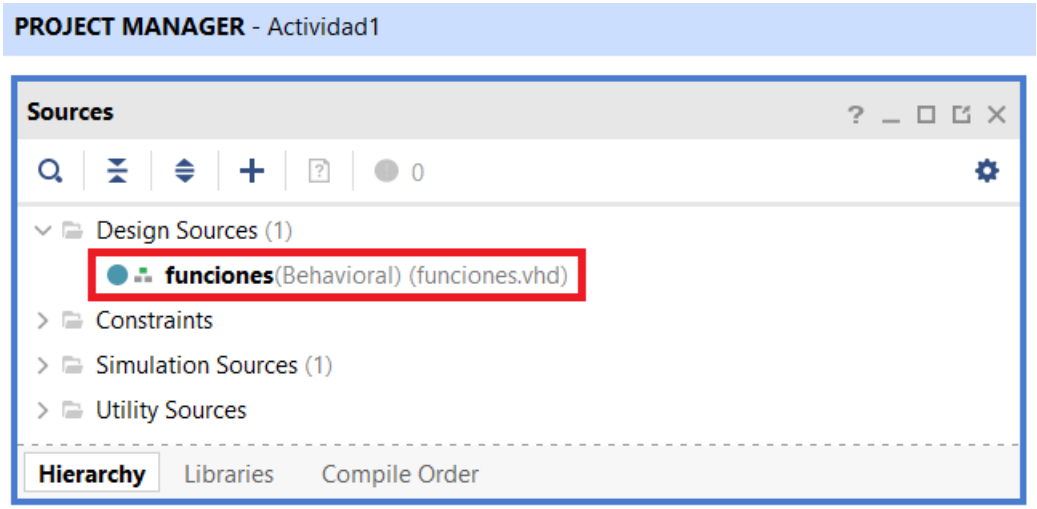


Figura 8. Captura de pantalla de *Sources* con el código generado resaltado con rectángulo rojo

El fichero generado se encuentra en la carpeta del proyecto cuya ruta se muestra en la Figura 9.

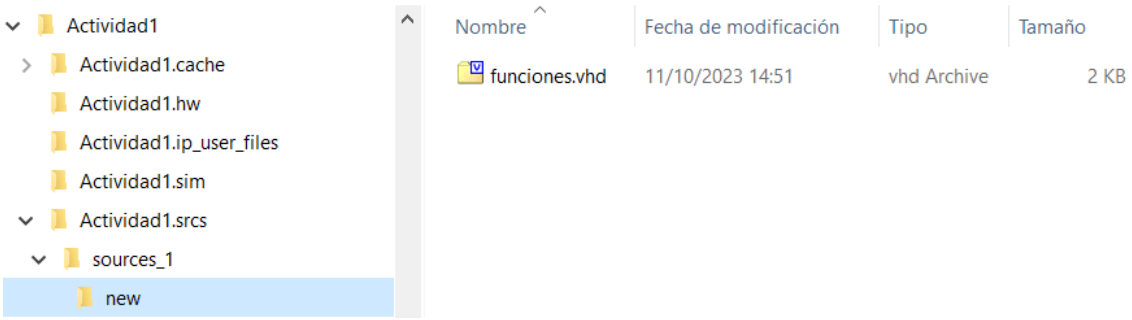


Figura 9. Captura de pantalla del explorador con la ruta del código VHDL generado con el asistente

4.- Estructura en un código VHDL

El asistente empleado en el apartado anterior genera una plantilla de un código VHDL. Haciendo doble clic en el nombre del fichero (cuadrado rojo en la Figura 8) se abre el código en el editor de texto de Vivado (Figura 10). Todo código VHDL está compuesto por 3 partes:

funciones.vhd

C:/Xilinx/MisProyectos/Actividad1/Actividad1.srcs/sources_1/new/funciones.vhd

```
1
2   library IEEE;
3   use IEEE.STD_LOGIC_1164.ALL;
4
5   entity funciones is
6       Port ( a : in STD_LOGIC;
7              b : in STD_LOGIC;
8              f0 : out STD_LOGIC;
9              f1 : out STD_LOGIC;
10             f2 : out STD_LOGIC;
11             f3 : out STD_LOGIC;
12             f4 : out STD_LOGIC;
13             f5 : out STD_LOGIC;
14             f6 : out STD_LOGIC);
15  end funciones;
16
17  architecture Behavioral of funciones is
18
19  begin
20
21
22  end Behavioral;
23
```

Figura 10. Captura de pantalla del explorador con la ruta del código VHDL generado con el asistente

- **Librerías** (rectángulo rojo). Siempre debe estar presente el paquete *IEEE.STD_LOGIC_1164* de la librería IEEE, que define, entre otros, los tipos *std_logic* y *std_logic_vector* para los puertos y señales; y los operadores básicos (*not*, *and*, *nand*, *or*, *nor*, *xor*, *xnor*). Como siempre debe estar presente, el asistente siempre incluye este paquete en la plantilla generada.
- **Entidad** (rectángulo verde). La entidad define la interfaz del circuito, es decir, las entradas y las salidas (con *Port*). Vivado genera la entidad con los datos que se introdujeron usando el asistente (Apartado 3). No obstante, se puede incluir o eliminar puertos editando la entidad si deseamos modificar la interfaz del circuito en un momento dado. Los puertos del mismo tipo se pueden escribir en la misma línea de

código separados por comas, pero se prefiere una línea por puerto por razones de estilo (mayor legibilidad).

Para declarar un puerto, éste debe tener un nombre, un modo (*in*, *out*) y un tipo. En este caso, el tipo *std_logic* soporta los valores lógicos 0 y 1, y otros valores que no usaremos en estas actividades [1].

- **Arquitectura** (rectángulo azul). En esta zona del código VHDL se describe la funcionalidad del circuito, que está vacía debido a que no aportamos en el asistente ninguna información al respecto. La arquitectura tiene un nombre que, por defecto, Vivado llama *Behavioral* (aunque puede usarse otro) y está asociado a la entidad (*funciones* en esta actividad).

  MUY IMPORTANTE: no se debe modificar el nombre de la entidad. Debe usarse el mismo nombre para el nombre del fichero, el nombre de la entidad y el nombre de la entidad a la que se asocia la arquitectura (líneas 5 y 17 en la Figura 10).

  A su vez, la arquitectura se divide en 2 partes:

  - **Zona declarativa**, antes de *begin* (rectángulo rosa). Aquí se declaran señales internas que se van a usar, así como componentes (los componentes vienen a ser subcircuitos). En esta actividad, el circuito es muy sencillo y no se va a declarar ninguno de estos objetos.
  - **Zona descriptiva o funcional**, después de *begin* (rectángulo naranja). En esta subsección del código, se describe la funcionalidad del circuito, es decir, lo que hace.

5.- Descripción del circuito en VHDL

La descripción del circuito a implementar en la FPGA se realiza en la zona descriptiva o funcional de la arquitectura (rectángulo naranja de la Figura 10).

Para describir el circuito de la Figura 1 se realiza empleando sus ecuaciones lógicas descritas en la sintaxis de VHDL. La arquitectura resultante del código VHDL se muestra en la Figura 11.

Cada una de las líneas de código añadidas es una **asignación concurrente** (se le asigna a cada puerto de salida el resultado de una puerta lógica en este caso). Para realizar una asignación, se usa <=. Por ejemplo, en la línea 22 se lee, de manera formal, que a *f1* se le asigna la salida de la puerta *and* cuyas entradas son *a* y *b*; y así con todas las demás.

```
17 architecture Behavioral of funciones is
18
19 begin
20
21     f0 <=    not a;
22     f1 <= a  and b;
23     f2 <= a nand b;
24     f3 <= a   or b;
25     f4 <= a  nor b;
26     f5 <= a  xor b;
27     f6 <= a xnor b;
28
29 end Behavioral;
```

Figura 11. Captura de pantalla de la porción del código VHDL que describe la funcionalidad (es decir, la arquitectura)

El lenguaje VHDL es meramente **concurrente**. Esto es, el orden en que se escriben las asignaciones en la zona funcional de la arquitectura es irrelevante, ya que el resultado final será un circuito implementado en FPGA donde todas las compuertas lógicas del circuito estarán operando simultáneamente. Por ejemplo, el código de la Figura 12 implementa el mismo circuito; y así para cualquier orden de las asignaciones.

```
17 architecture Behavioral of funciones is
18
19 begin
20
21     f6 <= a xnor b;
22     f1 <= a  and b;
23     f4 <= a  nor b;
24     f2 <= a nand b;
25     f3 <= a   or b;
26     f0 <=    not a;
27     f5 <= a  xor b;
28
29 end Behavioral;
```

Figura 12. Captura de pantalla de la porción del código VHDL que describe la funcionalidad con otro orden de las asignaciones concurrentes

Consideraciones a tener en cuenta:

- <u>No se puede asignar la salida de una puerta lógica a un puerto de entrada</u>. O, dicho de otra manera, los puertos de entrada no pueden

estar a la izquierda en las asignaciones concurrentes. La asignación de la Figura 13 da un error en la síntesis del circuito.

```
21      b  <=    not a;
22      f1 <= a  and
23      f2 <= a nand
24      f3 <= a   or b;
```

Error: cannot update object 'b' of mode 'in'

Figura 13. Captura de pantalla de asignación incorrecta a puerto de entrada

- Análogamente, no se puede asignar a la entrada de una puerta lógica a un puerto de salida. O, dicho de otra manera, los puertos de salida no pueden estar a la derecha en las asignaciones concurrentes. La asignación de la Figura 14 da un error en la síntesis del circuito.

```
21      f0 <=    not a;
22      f1 <= f0 and b;
23
24
```

Error: cannot read from 'out' object 'f0'; use 'buffer' or 'inout' instead

Figura 14. Captura de pantalla de asignación incorrecta a puerto de salida

- No está permitido realizar más de una asignación sobre el mismo puerto o señal (asignaciones múltiples). Una asignación como la que se muestra a la izquierda de la Figura 15 trata de inferir un circuito que se debe implementar, ya que no se puede conectar entre sí 2 ó más salidas de puertas lógicas y otros componentes digitales

```
f0 <=    not a;
f1 <= a  and b;
f2 <= a nand b;
f1 <= a   or b;
f4 <= a  nor b;
f5 <= a  xor b;
f6 <= a xnor b;
```

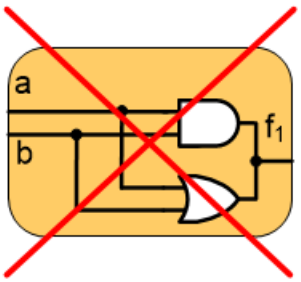


Figura 15. Captura de pantalla de un código con asignaciones múltiples y el circuito que trata de inferir

6.- Creación de un banco de pruebas (*test bench*)

La descripción del diseño finaliza con la introducción en el código VHDL de las asignaciones concurrentes del apartado 5. Pero eso sólo es la primera fase del diseño digital.

El siguiente paso necesario es comprobar que la lógica que se sintetiza realmente implementa lo que queremos. Este paso SIEMPRE es necesario, aun cuando la comprobación de la sintaxis y la síntesis no den problemas (supongamos que en la línea 24 del código de la Figura 11 escribimos *and* en lugar de *or*. No habrá errores, pero la función implementada NO es la deseada, y por tanto el diseño está mal, y esto no lo sabremos a no ser que simulemos y contrastemos los resultados con las especificaciones de partida).

Para comprobar la correcta descripción de las funciones y el correcto funcionamiento del circuito, hay que comprobar los valores de la tabla de verdad de las especificaciones de partida (Tabla 1).

Esta comprobación se realiza mediante un banco de pruebas, también llamado en inglés *test bench*. El *test bench* encapsula nuestro circuito VHDL en otro nivel jerárquicamente superior, tal como se ve en la Figura 16.

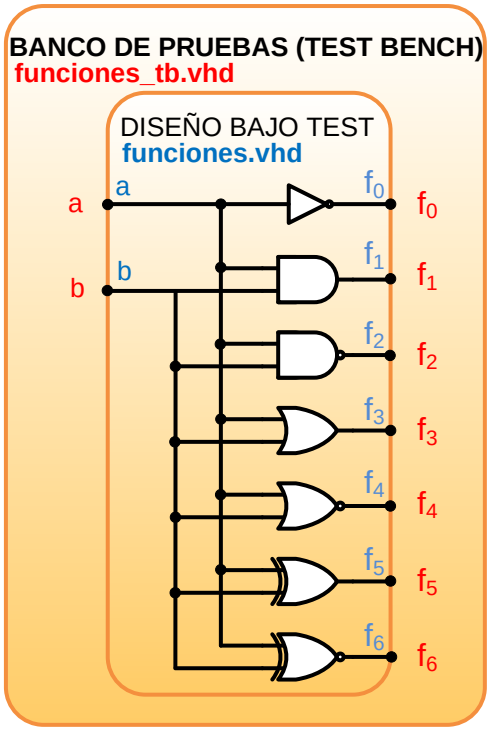


Figura 16. Esquema del banco de pruebas (test bench) incluyendo el circuito que se quiere testear (unidad bajo test)

VHDL permite descripciones que no son sintetizables, es decir, que no se pueden implementar directamente en un circuito electrónico digital, o, dicho de otra manera, sólo un subconjunto de VHDL es sintetizable (Figura 17). Este tipo de descripciones son útiles en simulación.

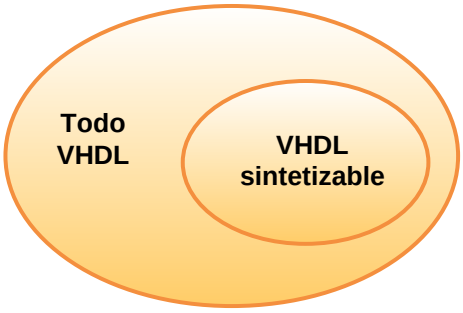


Figura 17. Sólo un subconjunto de VHDL es sintetizable

El *test bench* es un código VHDL NO sintetizable (no se puede implementar físicamente en la FPGA), en el que generamos estímulos a la entrada de la unidad bajo test (*unit under test – uut*) y observamos las salidas que provocan esos estímulos.

El *test bench* es un código VHDL, por lo que debe tener la estructura descrita en el apartado 4. No obstante, como se puede apreciar en la Figura 16, **el *test bench* carece de interfaz**, así que la entidad de este circuito no sintetizable aparece vacía, pues **no tiene puertos de entrada/salida**.

Para generar un *test bench* en Vivado, se procede de la siguiente manera:

1. Buscar en la barra de menús *File -> Add Sources*, o, más directamente, pulsar en el comando en el panel *Flow Navigator* (Figura 2).
2. Seleccionar *Add or create simulation sources* en el menú emergente que aparece (Figura 18).
3. En la siguiente ventana del asistente, pulsar en *Create File* (Figura 19).
4. A continuación, el asistente pide un nombre de fichero para el *test bench* que va a proceder a crear (Figura 20).

   NOTA: Es una buena costumbre poner un nombre al *test bench* que lo asocie al nombre del diseño que va a ser simulado. En esta práctica, si queremos simular el diseño de **funciones.vhd**, el *tech bench* debería llamarse **funciones_tb.vhd**, por ejemplo.

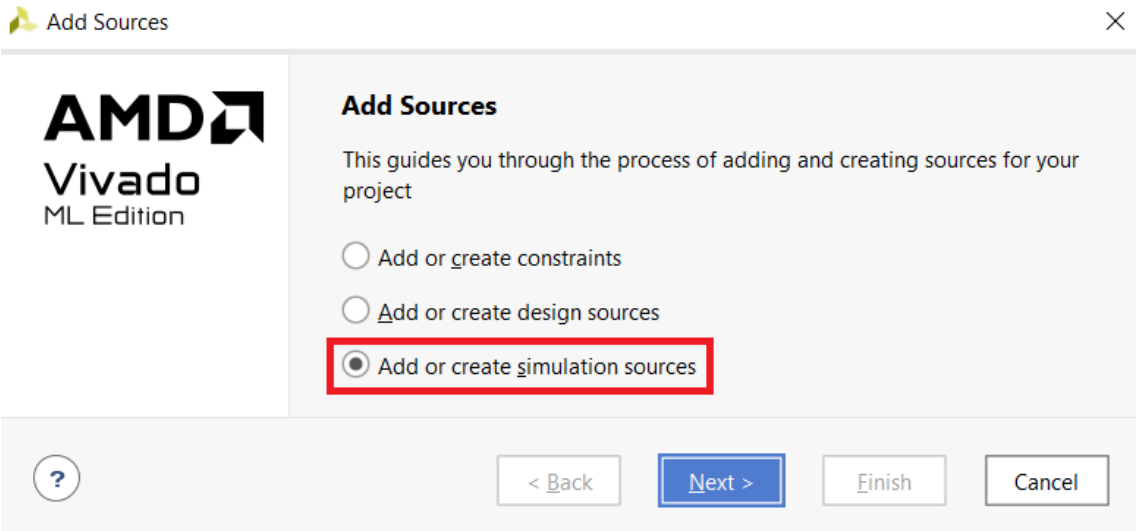


Figura 18. Captura de pantalla de *Add Source* con *Add or create simulation sources* resaltado con cuadro rojo

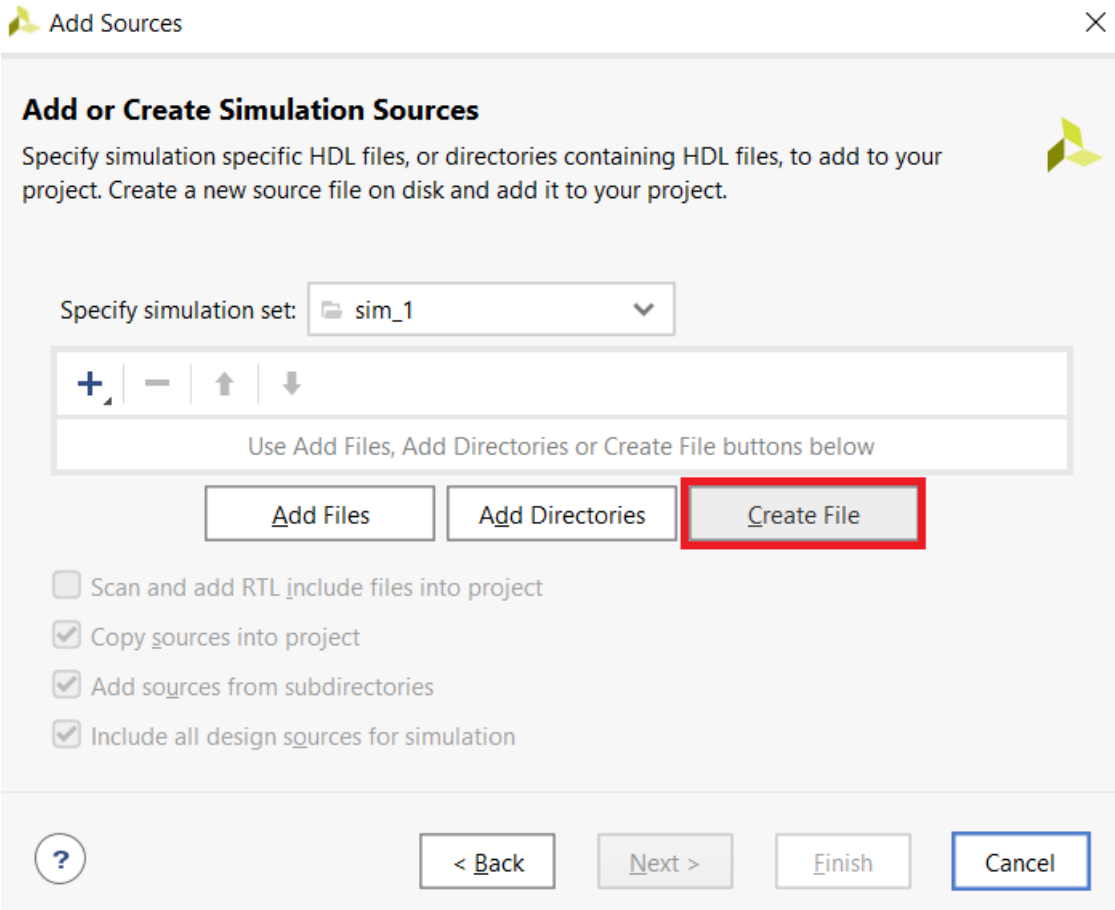


Figura 19. Captura de pantalla de *Add or Create Simulation Sourcse* con *Create File* resaltado con cuadro rojo

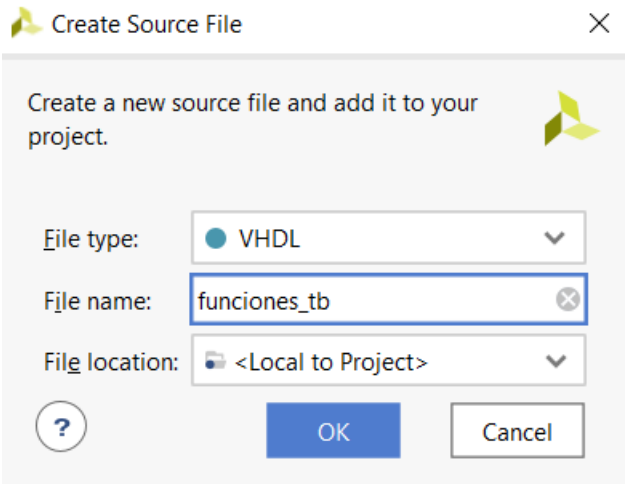


Figura 20. Captura de pantalla de *Create Source File* con el campo *File name* completado con el nombre del *test bench*

5. Una vez ingresado el nombre del test bench, éste aparecerá en la lista que aparece en la ventana *Add or Create Simulation Sources*. Hay que pulsar en *Finish* para terminar (Figura 21).

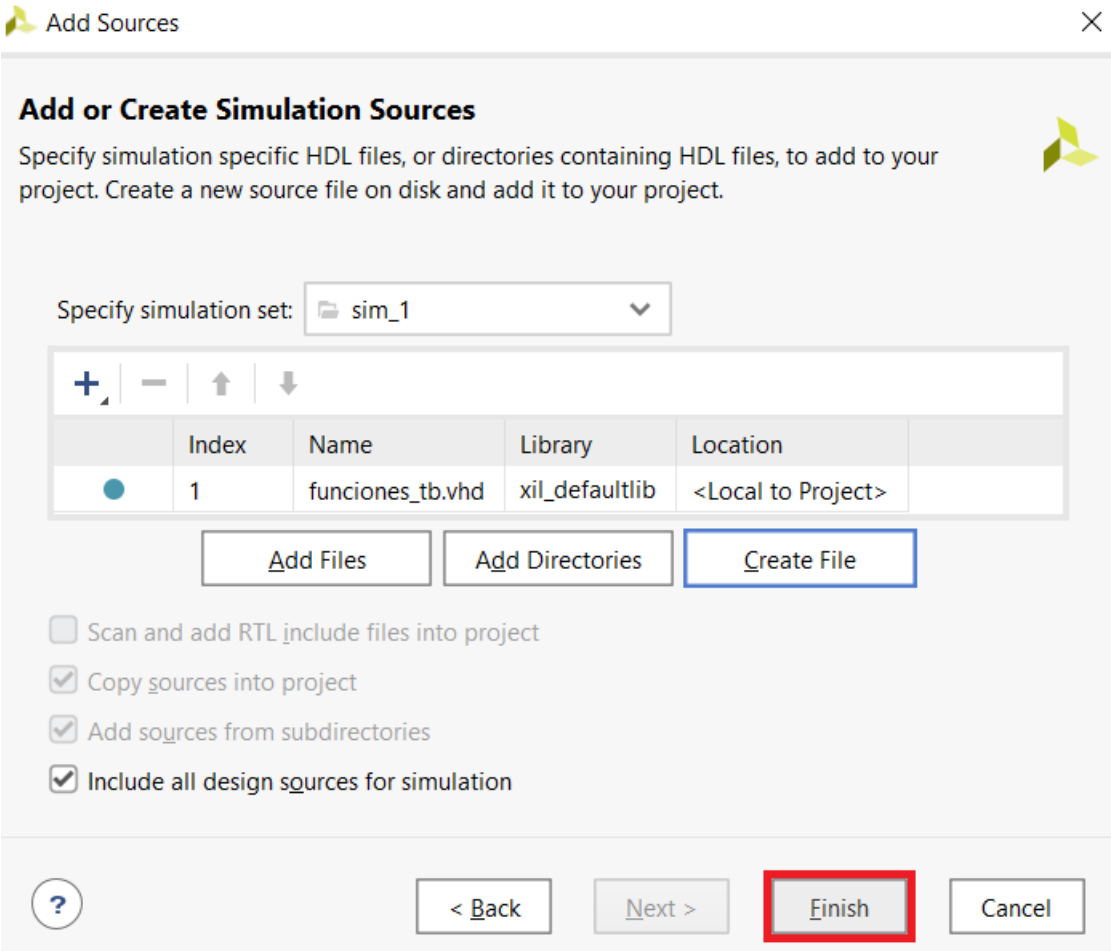


Figura 21. Captura de pantalla de *Add or Create Simulation Sources* con *Finish* resaltado con rectángulo rojo.

6. Por defecto, el asistente pide en el siguiente paso que se ingresen los puertos de entrada y salida (igual que en el apartado 3). Como se ha mencionado, el *test bench* no tiene puertos, así que no hay introducir nada en el campo *I/O Port Definitions* y pulsar en *OK* (Figura 22).

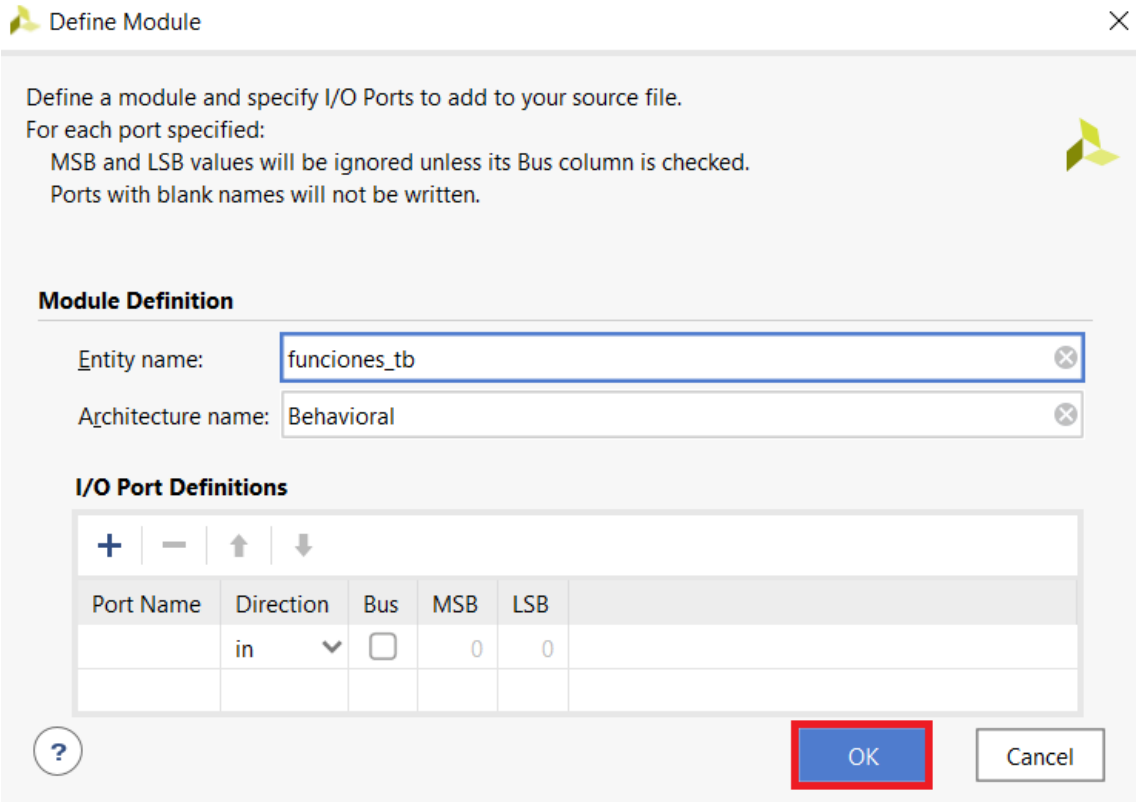


Figura 22. Captura de pantalla de *Define Module* con el campo *I/O Port Definitions* vacío.

7. Un mensaje previene de que no se ha modificado interfaz del módulo. Hacer clic en *Yes* (Figura 23).

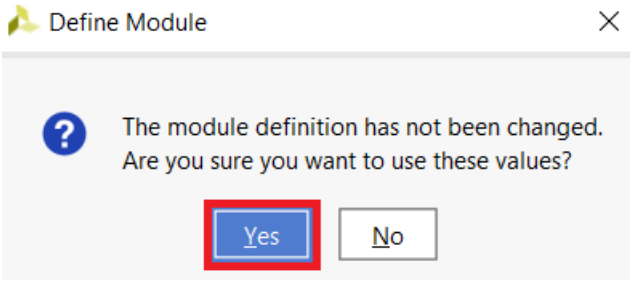


Figura 23. Captura de pantalla del aviso.

Al finalizar el último paso del asistente se crea un nuevo código VHDL, en este caso, del *test bench*. Este nuevo fichero puede verse en la ventana de *Sources*, bajo la carpeta de ficheros de simulación, *sim_1* (ver Figura 24).

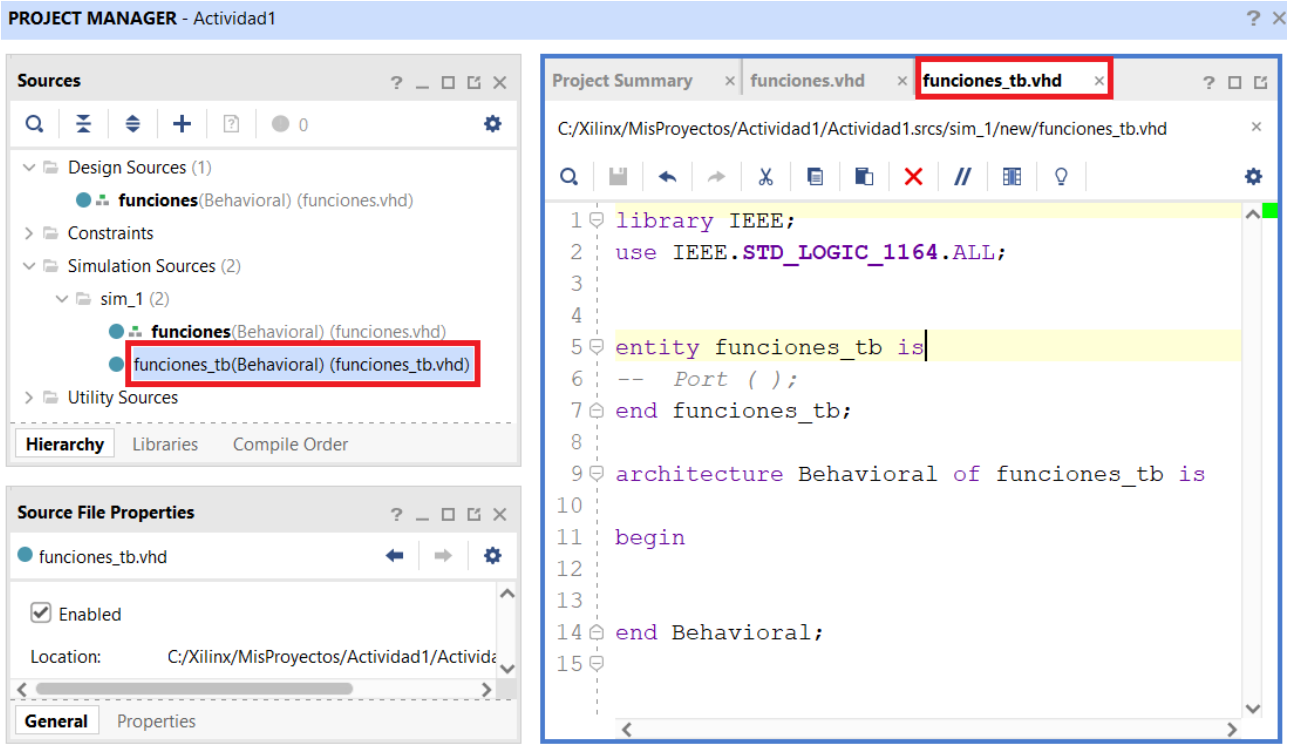


Figura 24. Captura de pantalla de Vivado con el test bench abierto en el editor.

El código del *test bench* se muestra en el editor de texto de Vivado haciendo doble clic en *funciones_tb* (Figura 24). Se puede apreciar que la entidad no tiene puertos declarados.

A continuación, es necesario modificar el *test bench* para incluir el circuito que hay simular, además de realizar el conexionado (mapeo) adecuado de las señales del *test bench* con los puertos del circuito a simular, según la Figura 16.

El código completo del *test bench* se muestra en la Figura 25. Para agilizar la escritura del *test bench*, se puede usar algunos generadores que figuran en la web [2 ,3]. Estos generadores son genéricos, y únicamente será necesario emplear la porción de código que generan que se muestra en la Figura 25. Como todo código VHDL, consta de 3 partes:

- Librerías (líneas 1 y 2)
- Entidad (líneas 4 y 5), sin puertos de entrada/salida definidos
- Arquitectura (líneas 7 en adelante). La zona declarativa de la arquitectura está comprendida hasta la palabra *begin* (línea 35) y la zona funcional, de ahí hasta el final del código.

A continuación, se procede a describir cada una de las partes de la que está compuesta la arquitectura de la figura 25, tanto en su parte declarativa como en su parte funcional.

Figura 25. Código del *test bench* funciones_tb.vhd.

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity funciones_tb is
5  end funciones_tb;
6
7  architecture tb of funciones_tb is
8
9      component funciones
10         port (a  : in std_logic;
11               b  : in std_logic;
12               f0 : out std_logic;
13               f1 : out std_logic;
14               f2 : out std_logic;
15               f3 : out std_logic;
16               f4 : out std_logic;
17               f5 : out std_logic;
18               f6 : out std_logic);
19     end component;
20
21     signal a  : std_logic := '0';
22     signal b  : std_logic := '0';
23     signal f0 : std_logic;
24     signal f1 : std_logic;
25     signal f2 : std_logic;
26     signal f3 : std_logic;
27     signal f4 : std_logic;
28     signal f5 : std_logic;
29     signal f6 : std_logic;
30
31     constant Tb_semiPeriod : time := 10 ns;  -- Put right period here
32
33     signal TbSimEnded : std_logic := '0';
34
35 begin
36
37     dut : funciones
38     port map (a  => a,
39               b  => b,
40               f0 => f0,
41               f1 => f1,
42               f2 => f2,
43               f3 => f3,
44               f4 => f4,
45               f5 => f5,
46               f6 => f6);
47
48     -- Stimuli generation
49     b <= not b after Tb_semiPeriod   when TbSimEnded /= '1' else '0';
50     a <= not a after Tb_semiPeriod*2 when TbSimEnded /= '1' else '0';
51
52     -- End of simulation: multiply Tb_semiPerior by 2^nº variables
53     TbSimEnded <= '1' after 4 * Tb_semiPeriod;
54 end tb;
```

En la zona de declarativa de la arquitectura se encuentran:

- La declaración del componente (rectángulo azul). Se declara el circuito (componente) que se va a usar. En la sintaxis debe constar junto con la palabra clave *component* el nombre de la entidad del circuito que se va a simular (***funciones***, en este caso). A continuación, los puertos de entrada/salida de este circuito (los mismos que la entidad en el código **funciones.vhd**).
- La declaración de las señales internas del *test bench* (rectángulo rojo). *El test bench* carece de puertos en la entidad, pero se definen unas señales que se conectarán al circuito bajo test. Las señales internas *a* y *b* están inicializadas con valor 0 lógico, ya que serán conectadas a los puertos de entrada *a* y *b* del circuito bajo test. El resto de señales serán conectadas a los puertos de salida del circuito (por eso no tienen un valor inicial), pudiendo observar los valores obtenidos los valores que se ingresan a través de *a* y *b*.
- Señales y constantes de simulación (rectángulo verde). La constante *Tb_semiPeriod* define un tiempo arbitrario (10 *ns*, por ejemplo), que se usará para modificar señales internas *a* y *b*. La señal interna *Tb_simEnded* se usará para controlar hasta qué momento cambiarán *a* y *b* en función del tiempo definido.

Cabe destacar que el orden en que aparecen todas estas definiciones en el código es irrelevante mientras estén en el ámbito de la zona declarativa de la arquitectura. Por ejemplo, la declaración de las señales podría encontrarse antes en el código que la declaración del componente.

Por otra parte, en la zona de funcional de la arquitectura se encuentran:

- El mapeo de puertos, *port map* (rectángulo naranja). El *port map* realiza una conexión de los puertos del circuito que se va a ser objeto de simulación con las señales internas del *test bench* que se han definido en la zona declarativa de la arquitectura (líneas 21 a 29). Para realizar el mapeo se usa el operador **=>** (no confundir con el operador usado en las asignaciones, <=).

  Normalmente, en los *test bench*, el mapeo se realiza con objetos que tienen el mismo nombre, aunque su ámbito sea distinto, lo que puede llevar a confusión:

  - Los objetos a la izquierda del mapeo (marcados en azul en el *port map*) se corresponden a los puertos de entrada/salida del circuito a testear (marcados en azul en el circuito).
  - Los objetos a la derecha del mapeo (marcados en rojo en el *port map*) se corresponden a las señales internas del *test bench* (marcados en rojo en el circuito).

Al comienzo del *port map* hay que poner una etiqueta. En principio puede tener cualquier nombre, aunque en los *test bench* suele usarse *dut*, acrónimo de *design under test*; o *UUT*, acrónimo de *Unit Under Test*.

- Generación de estímulos (rectángulo violeta). En principio no sería necesaria esta parte, ya que se ha introducido unos valores de entrada en las líneas 21 y 22; y al realizar la simulación se obtienen los valores de salida consecuentes con los valores de entrada. No obstante, en circuitos combinacionales como éste, hay que comprobar todas las combinaciones de las entradas de acuerdo con la Tabla 1. En este sentido, las líneas 49 y 50 modifican los valores de entrada de manera adecuada para cubrir todas las combinaciones, como se aprecia en el cronograma. En este caso, con 10 *ns* definidos como tiempo de cambio para la variable menos significativa, supone que en 40 *ns* ya se ha realizado todas las combinaciones, de tal manera que las señales *a* y *b* se quedan indefinidamente a *0* lógico desde ese momento. Las asignaciones de las líneas 21 y 22 no son sintetizables, esto es, no se pueden traducir a hardware (ver Figura 17).

El empleo del circuito descrito en *funciones.vhd* en el banco de pruebas *funciones_tb.vhd* a través del *port map* hace que el panel de *Sources* tenga la apariencia mostrada en la Figura 26. Puede apreciarse que el test bench *funciones_tb* es el fichero activo para simulación (icono) y en su interior se encuentra integrado el circuito *funciones*, que será objeto de test con él, de acuerdo con la Figura 16.

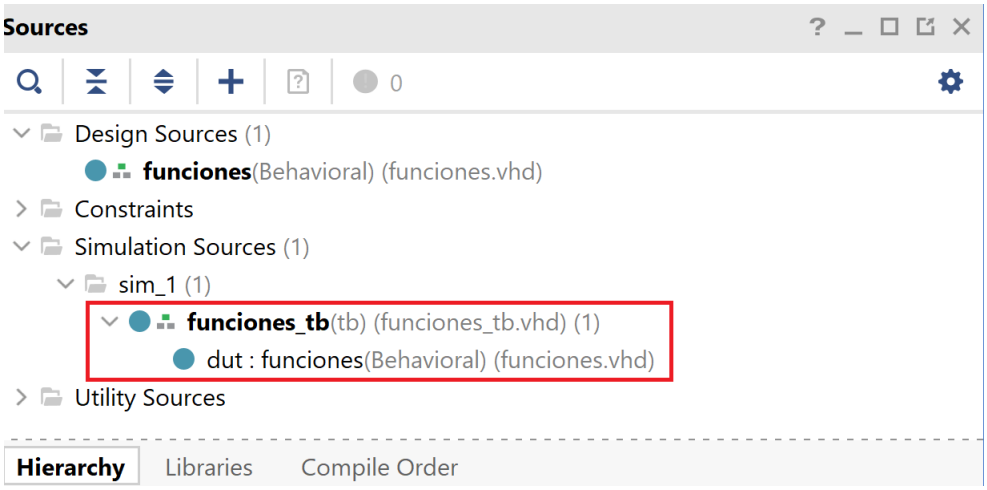


Figura 26. Captura de pantalla de *Sources* mostrando la jerarquía en *Simulation Sources*.

El fichero del *test bench* generado se encuentra en la carpeta del proyecto cuya ruta se muestra en la Figura 27.

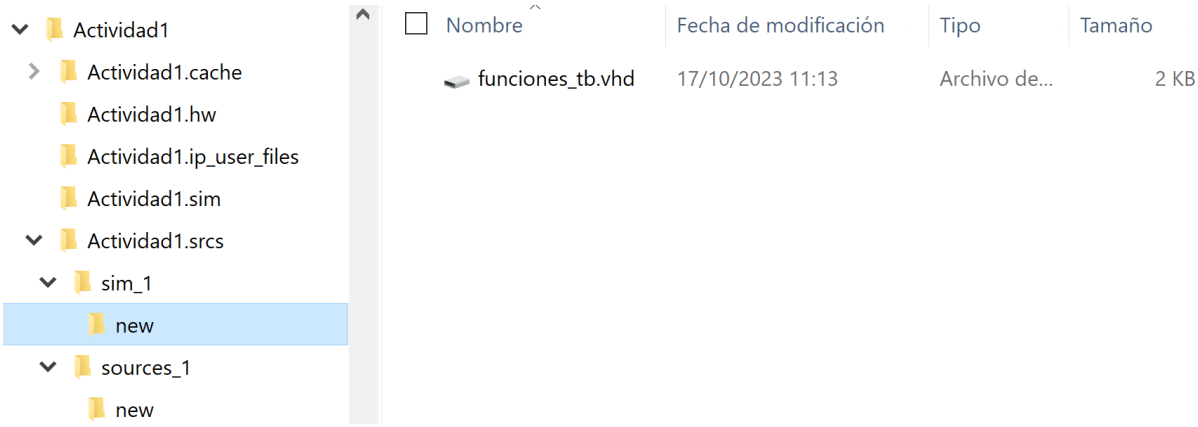


Figura 27. Captura de pantalla del explorador con la ruta del código VHDL del *test bench* creado.

7.- Simulación del circuito

Para simular el circuito, hay que hacer clic en *Run Simulation* en el panel *Flow Navigator*. Esto abrirá una ventana emergente en la que hay que pulsar *Run Behavioral Simulation* (ver Figura 28). Esto hará que se ejecute la simulación para el *test bench* activo en el panel de *Sources*.

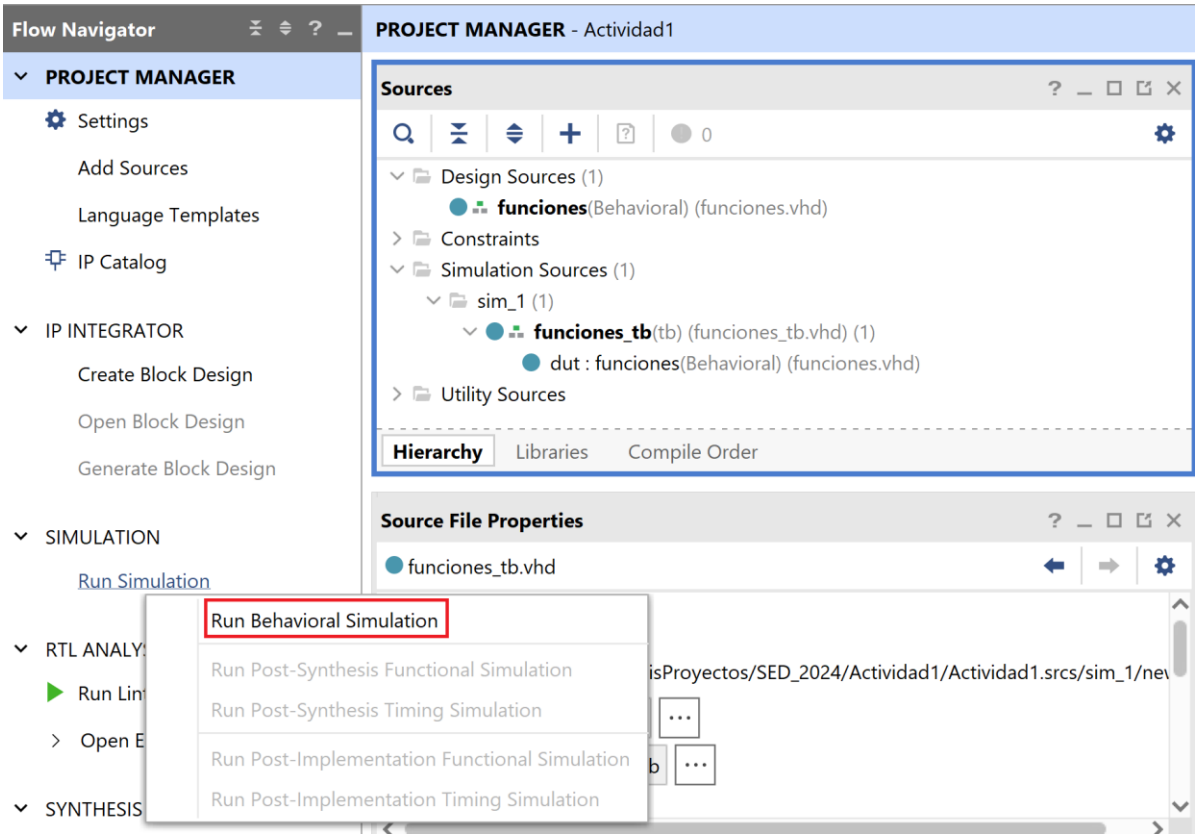


Figura 28. Captura de Vivado con *Run Behavioral Simulation* recuadrado en rojo.

El simulador de Vivado, denominado *Vivado Simulator*, crea un cronograma temporal con los estímulos que fueron definidos en el *test bench*. La apariencia de este cronograma se muestra en la Figura 29.

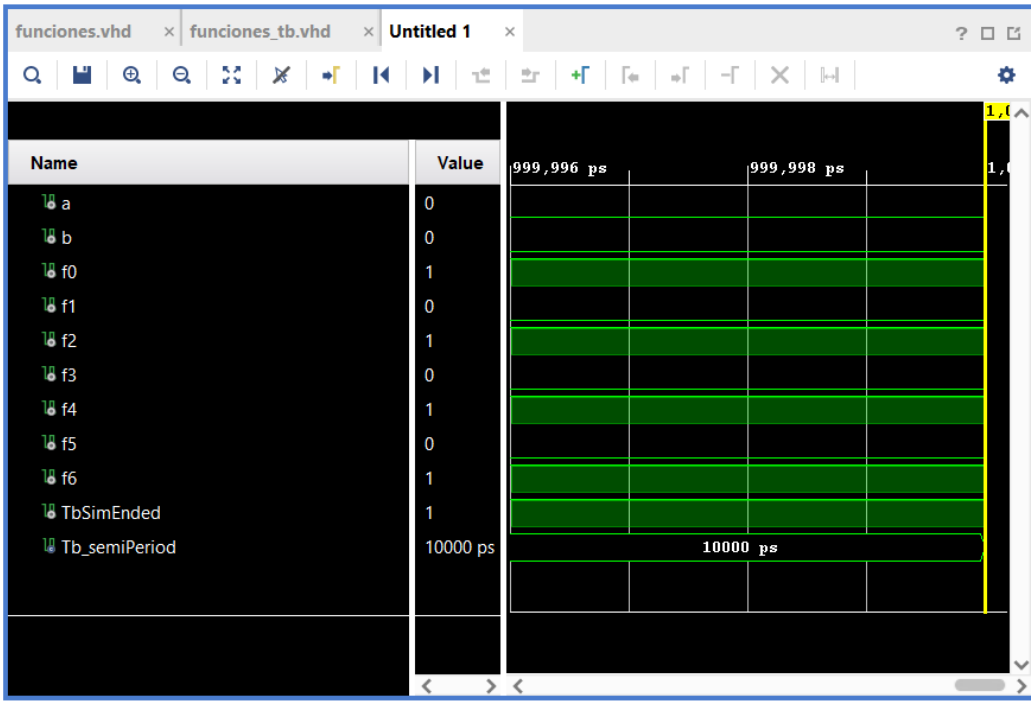


Figura 29. Captura del cronograma de simulación generado.

Es necesario ajustar la escala de representación del cronograma temporal para una visualización óptima. Se recomienda proceder de la siguiente manera:

1. Ajustar el cronograma a todo el tiempo de simulación usando el comando *Zoom Fit* (icono de la barra de herramientas del cronograma).
2. Usar las lupas ( ) y la barra de desplazamiento horizontal para ajustar la escala y visualizar la región de interés del cronograma a la escala adecuada. Además, también es posible desincrustar el cronograma de la ventana principal de Vivado para aprovechar toda la pantalla del monitor para ver el cronograma, usando el icono .

En la Figura 30 se muestra una visualización posible. En la columna de la izquierda aparece el nombre de las señales, en la columna central el valor que toman las señales en un tiempo de simulación de terminado por un marcador; y a la derecha se muestran las formas de onda, los valores que van tomando las señales a lo largo del tiempo de simulación. Cuando el nivel está alto, la señal tiene un valor de *1* lógico; y cuando el nivel está bajo, la señal tiene un valor de *0* lógico.

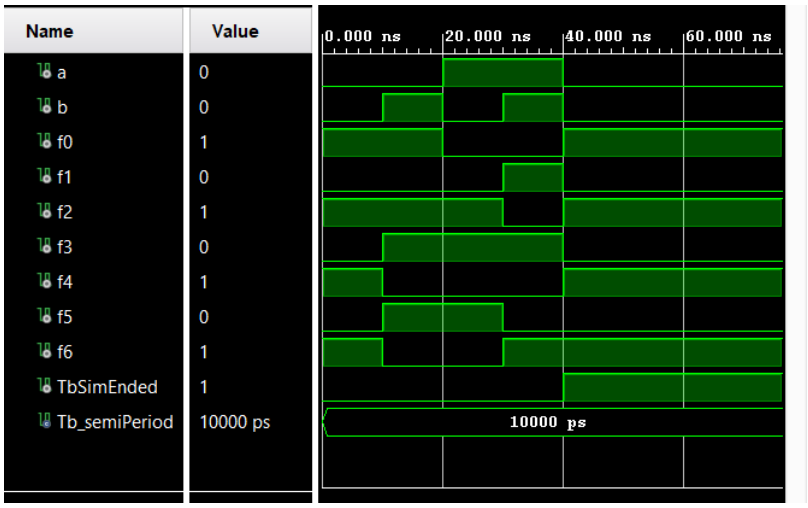


Figura 30. Captura del cronograma de simulación ajustado.

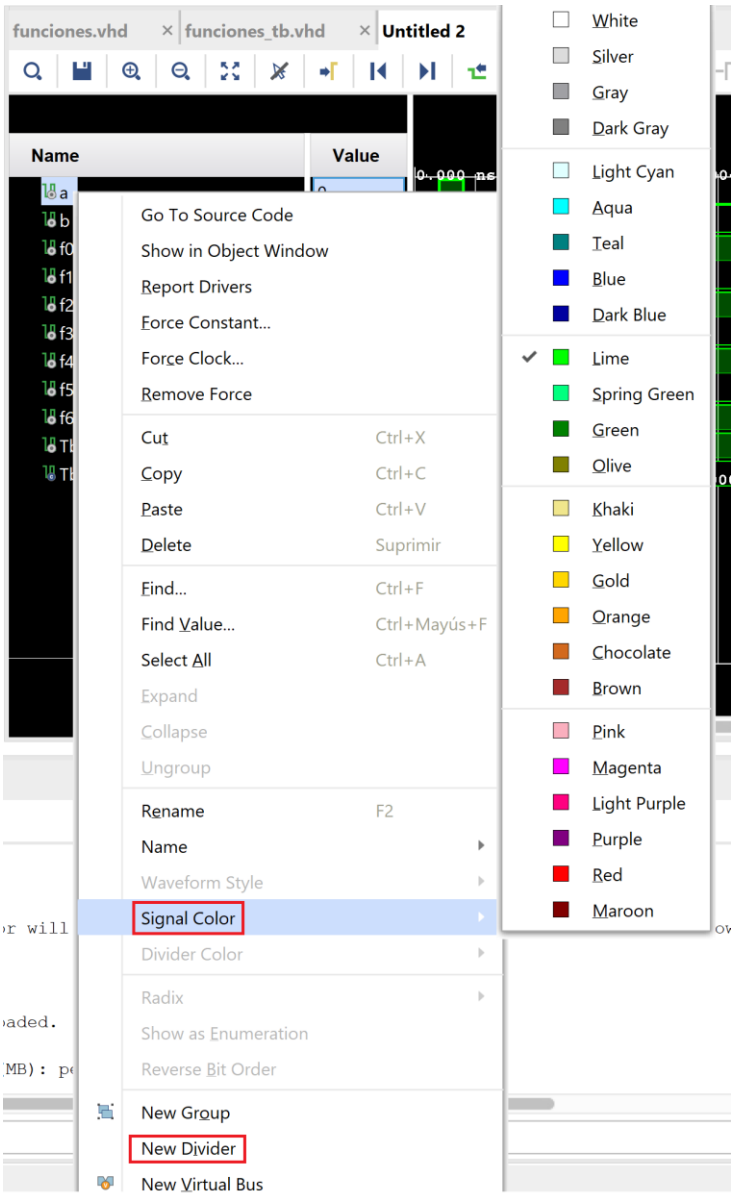
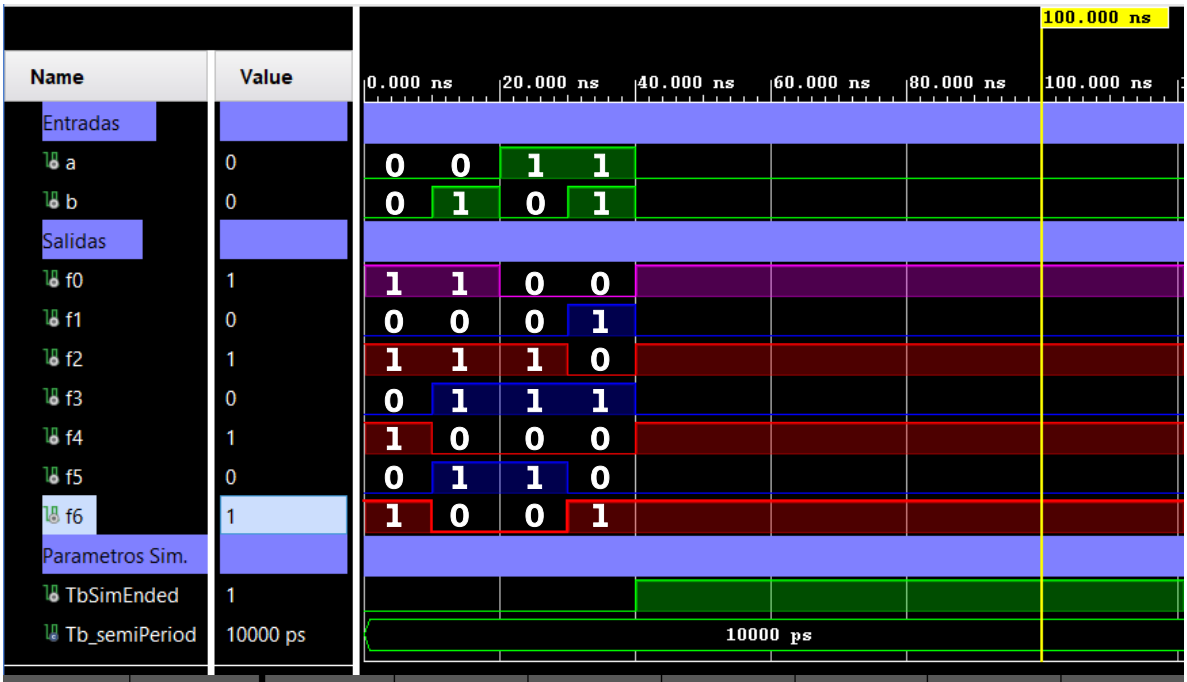


Figura 31 Captura del menú emergente del simulador.

Es posible seguir mejorando la visualización del cronograma añadiendo divisores entre las señales de entrada, de salida y de simulación; y modificando los colores de las señales. Para poder realizar esto, hay que pulsar con el botón derecho del ratón sobre el nombre de la señal sobre la que desee crear un divisor o cambiar el color. Operando de esta manera se abre el menú emergente de la Figura 31, donde están marcados en rojo los comandos que se van a emplear. Se puede modificar la fila de los divisores arrastrándolos con el ratón con el botón izquierdo pulsado.

El resultado final del formateo del cronograma se muestra en la Figura 32, donde se ha incorporado los valores numéricos a las señas, así como la Tabla 1 por conveniencia. De esta manera, se puede comprobar fácilmente que el circuito responde a la tabla de verdad de partida y que está correctamente diseñado.



a	b	f_0	f_1	f_2	f_3	f_4	f_5	f_6
0	0	1	0	1	0	1	0	1
0	1	1	0	1	1	0	1	0
1	0	0	0	1	1	0	1	0
1	1	0	1	0	1	0	0	1

Figura 32 Captura del cronograma de simulación ajustado y formateado.

El formateo del cronograma resulta tedioso si hay que realizar muchas simulaciones sobre un circuito. Por eso, interesa guardar el fichero de configuración del cronograma. Para ello, debemos hacer uso del comando *Save Waveform Configuration* en la barra de herramientas (Figura 33).

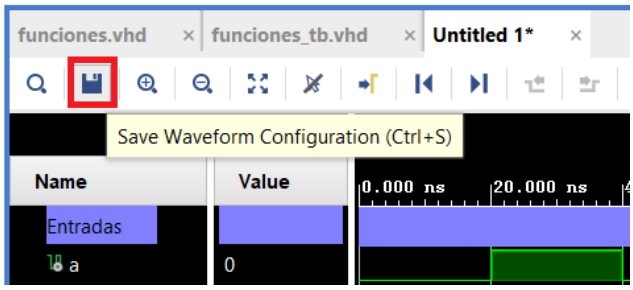


Figura 33 Captura pantalla con el comando *Save Waveform Configuration* resaltado.

Esto crea un fichero de extensión *.wcfg*. Si incluimos la opción que lo añade el proyecto, podremos verlo en la ventana de *Sources* (Figura 34).

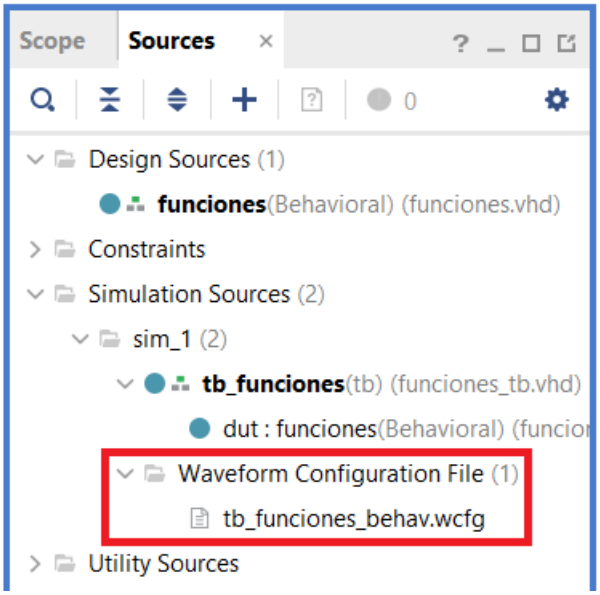


Figura 34 Captura pantalla de la ventana Source con el archivo de configuración del cronograma resaltado en rojo.

8.- Cerrar el proyecto y recuperar los ficheros importantes

Para cerrar el proyecto hay que buscar la ruta File -> Project -> Close. El acceso se muestra en la Figura 35.

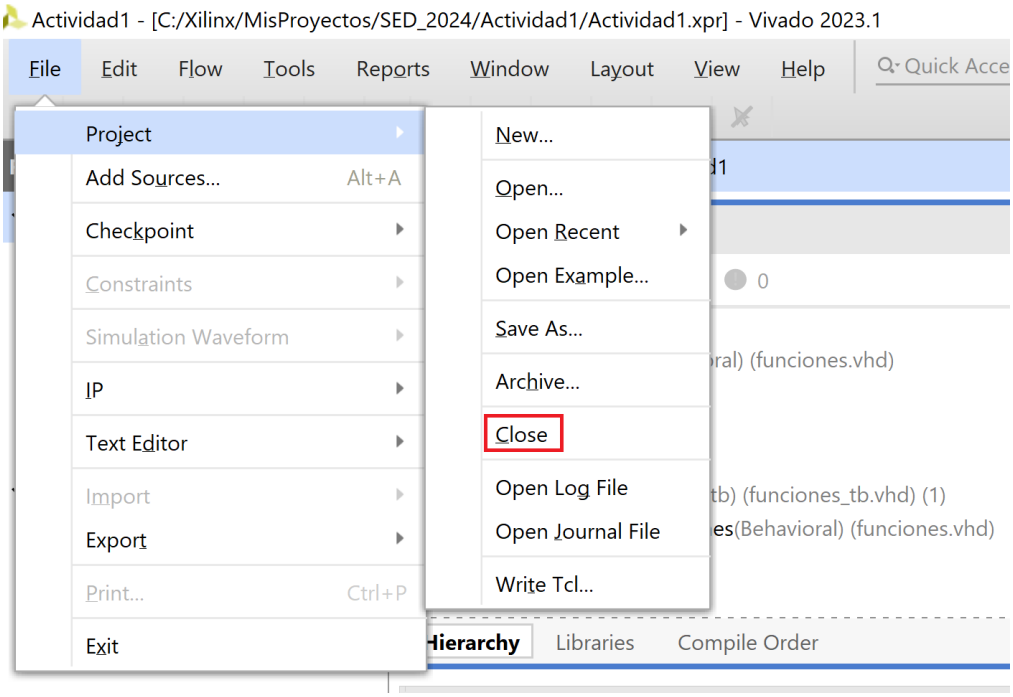


Figura 35 Captura pantalla del acceso al comando *Close Project*.

Los ficheros importantes, que son los códigos VHDL de los circuitos y los testbench, se encuentran en la carpeta *Actividad1.srcs* (*srcs* es una abreviatura de *sources*, fuentes en inglés). La ruta se muestra en la Figura 36.

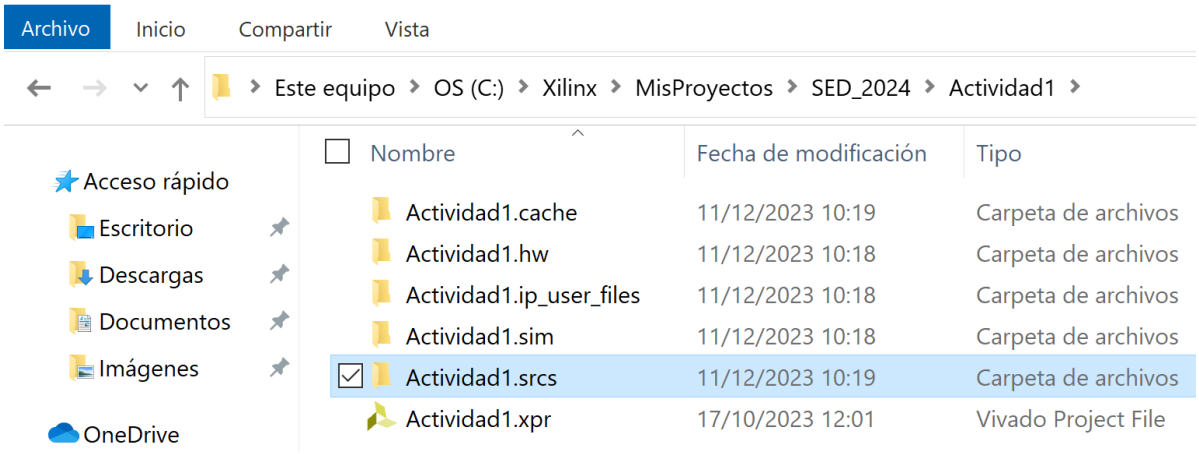


Figura 36 Captura pantalla la carpeta del proyecto *Actividad1*.

Referencias

[1] https://www.hdlworks.com/hdl_corner/vhdl_ref/VHDLContents/StdLogic1164.htm
[2] https://vhdl.lapinoo.net/testbench/
[3] https://www.doulos.com/knowhow/perl/vhdl-testbench-creation-using-perl/